METHODOLOGY DOCUMENT

Cognitive Decision Intelligence Framework for Dynamic Business Strategy Using AI

Implementation methodology — Draft

1. End-to-end workflow

1) Acquire the authorized dataset files and record their source/version.
2) Validate required columns, types, missingness, duplicates, and target definitions.
3) Clean and transform data using dataset-specific procedures.
4) Engineer predictors using only information available at prediction time.
5) Create leakage-aware train/validation/test partitions.
6) Fit baseline and advanced models.
7) Evaluate using task-appropriate metrics.
8) Export tidy metric records and prediction outputs.
9) Consolidate results and visualize them in Streamlit.

2. Walmart sales forecasting

Merge train.csv with features.csv on Store, Date, and IsHoliday; join stores.csv on Store. Parse Date and sort records chronologically. Create calendar variables such as year, month, week, and holiday indicator. Handle optional markdown fields carefully. Candidate models include Linear Regression, Random Forest, and XGBoost. Use a chronological holdout; report MAE, RMSE, and R^2. If lag features are used, construct them within each Store/Dept series using past values only.

3. Amazon review sentiment classification

Identify the rating and review-text fields in the source release. Remove missing text and invalid ratings. If using binary sentiment, explicitly document the mapping (for example, 1–2 negative, 4–5 positive, 3 excluded). Split with stratification. Fit TF-IDF vectorization inside each model pipeline. Compare Logistic Regression, Multinomial Naive Bayes, and Linear SVM. Report accuracy, precision, recall, F1, and a confusion matrix. Inspect class balance and error examples.

4. M5 demand forecasting

Convert daily sales columns from wide to long format, join calendar fields by day identifier, and join prices using store/item/week keys. Begin with a documented subset of item/store series if memory is constrained. Sort each series by date. Construct lag and rolling features by shifting the target before calculating rolling statistics. Use chronological validation. Compare XGBoost, LightGBM, and LSTM only when all models use equivalent series, forecast horizon, and validation periods.

5. Unified evaluation schema

Store one record per dataset/model/metric using the columns Dataset, Model, Metric, and Value. Optional fields include Split, Run_Date, Sample_Size, Horizon, and Notes. The unified table enables dashboard filtering, but comparisons must remain within compatible tasks and metrics.

6. Reproducibility and quality controls

Set and record random seeds where applicable. Record Python and package versions, hardware, data source/version, sampling, split dates, and preprocessing choices. Fit preprocessing only on training data. Keep test data untouched until final evaluation. Save model artifacts with their feature schema and metadata. Do not load untrusted pickle/joblib files. Never commit credentials or restricted datasets.

7. Model artifact policy

Only save model artifacts after training on the intended authorized dataset. For text classification, save the complete fitted pipeline (TF-IDF plus classifier), not only the classifier. For forecasting, save the fitted preprocessing/model pipeline and the feature-column order. Include metadata such as training timestamp, dataset version, split, metrics, and package versions. Large artifacts may be better stored in Git LFS or a model registry.

8. Final reporting checklist

- ■ Same validation protocol for models within a task.
- ■ No target leakage in engineered features.
- ■ All model metrics generated from actual predictions.
- ■ Model artifact reload tested.
- ■ Dashboard values trace back to saved results.
- ■ Dataset license and attribution included.